calculated MD5 value for it; `md5_file()` takes a string, which is evaluated to a file path, which is read into memory and processed in the same way (assuming the file exists and is readable for the PHP process).

The main reason `md5()` is not appropriate these days for cryptography is because of its speed: it's very quick on modern hardware. We'll cover what hash rate means for cryptography in an upcoming section in this chapter; just know that a single mid-tier GPU such as the RTX 3060 can calculate at a rate in the order of 10^4 MH/s. (That's tens of thousands of mega-hashes per second – $10^4 * 10^6 = 10^{10}$ hash values *per second*). Those md5 "encrypted" user passwords in your databases aren't looking so safe anymore….

PHP's `md5()` function returns a 32-character string by default, which is a hexadecimal representation of the 16-byte binary value that the algorithm produces. Alternatively, you can get the raw binary directly:

```
print md5('some data') . PHP_EOL;
print bin2hex(md5('some data', true)); // returns binary value
```

```
Output:
1e50210a0202497fb79bc38b6ade6c34
1e50210a0202497fb79bc38b6ade6c34
```

sha1(), sha_file()

Very similar in operation to the MD5 functions above, `sha1()` and `sha1_file()` also subject string (or file) data to a hashing algorithm; this time it is the US Secure Hashing Algorithm 1. And much like MD5, it is no longer considered useful for cryptography due to its speed.

```
print sha1('some data') . PHP_EOL;
print bin2hex(sha1('some data', true)); // returns binary value
```

```
Output:
baf34551fecb48acc3da868eb85e1b6dac9de356
baf34551fecb48acc3da868eb85e1b6dac9de356
```

As you can see, `sha1` returns a longer hash value than `md5()`, which means that collision probability is lowered accordingly. We'll revisit hashing later in the chapter, in the section on cryptography.